

# Convolutional Neural Networks

---

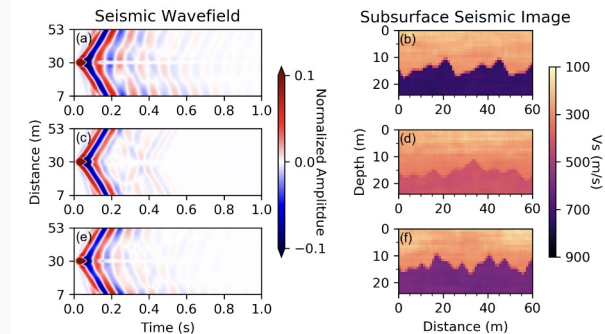
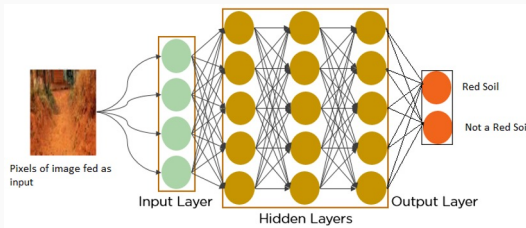
**PROF. KRISHNA KUMAR, PHD**

J. Neils Thompson Centennial Teaching Fellow

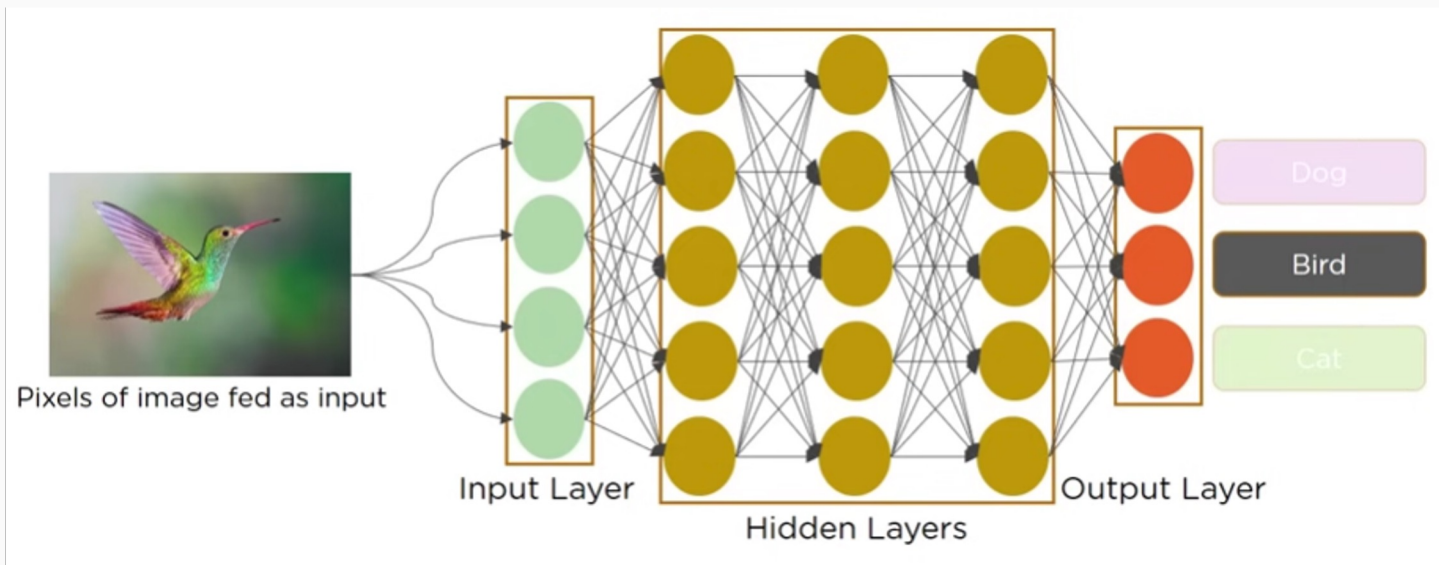
Department of Civil, Architectural, and Environmental Eng.  
The University of Texas at Austin

# Learning Objectives

- What is a Convolutional Neural Networks (Image-based data)
- CNN Architecture
- Uses of CNN:
  - Image classification (Soil Classification/Crack detection)
  - Image segmentation (Mapping cracks/Soil layer separation)
  - Image generation (Generate soil profile based on measurements)

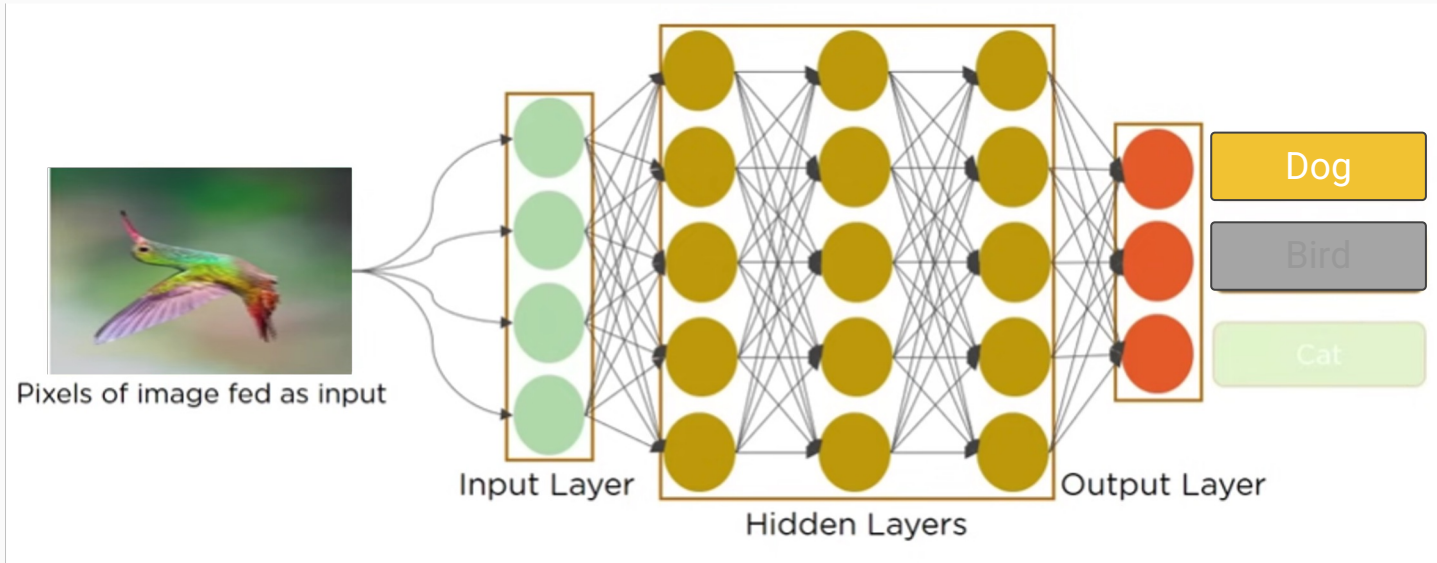


# Why do we need CNN?



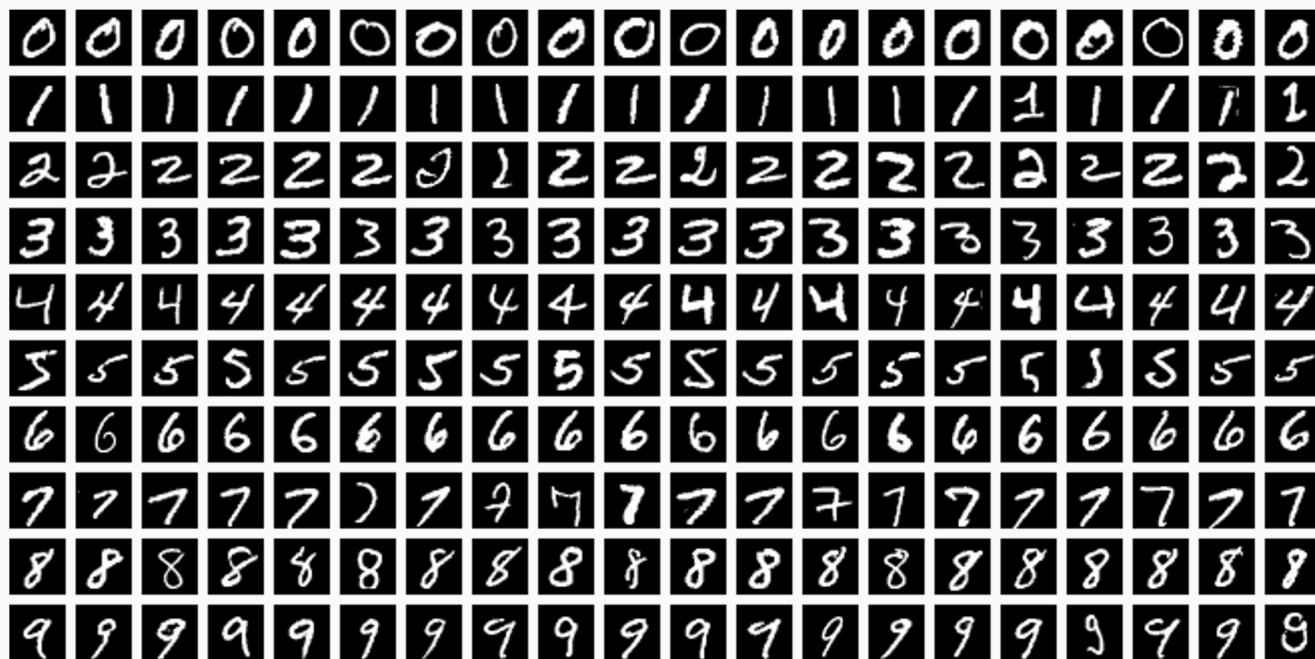
Multi- Layer Perceptron

# Why do we need CNN? (MLP Failure)

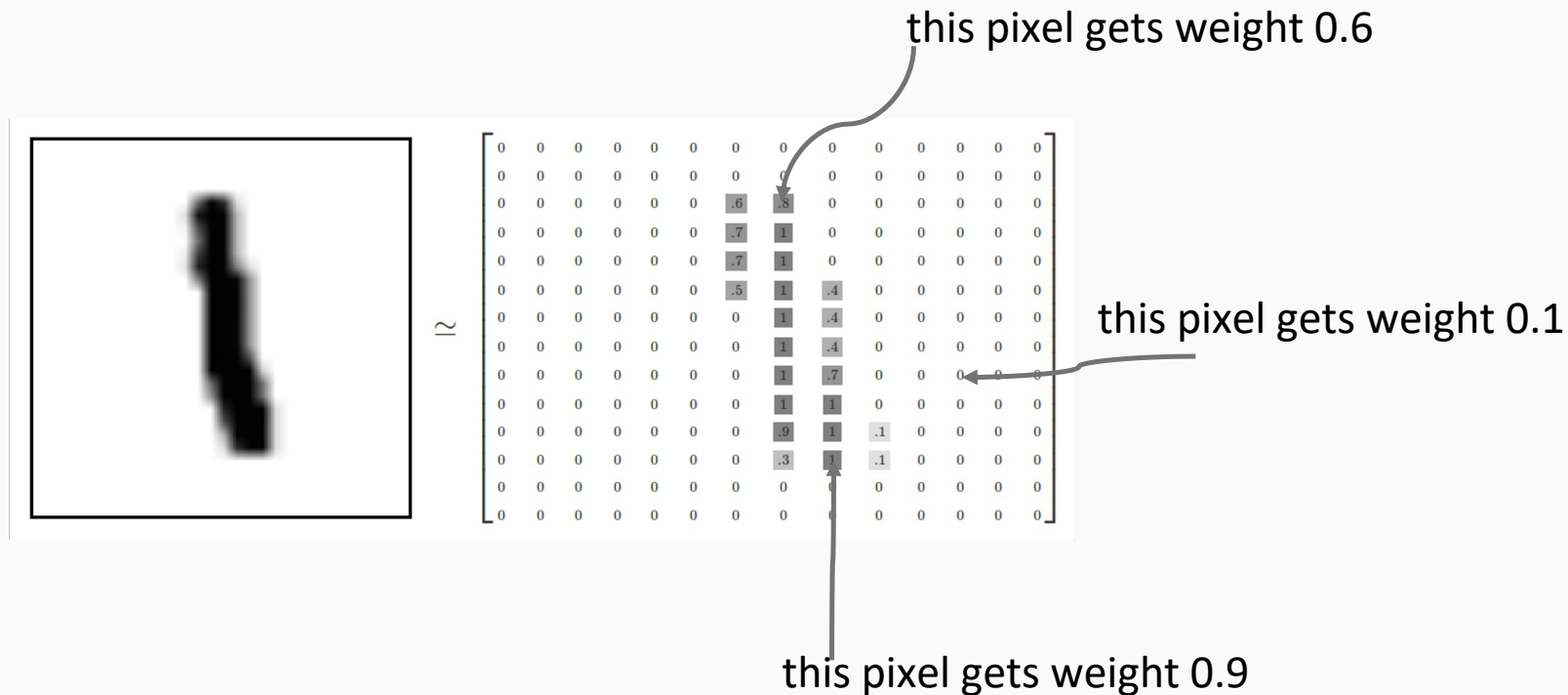


Multi- Layer Perceptron

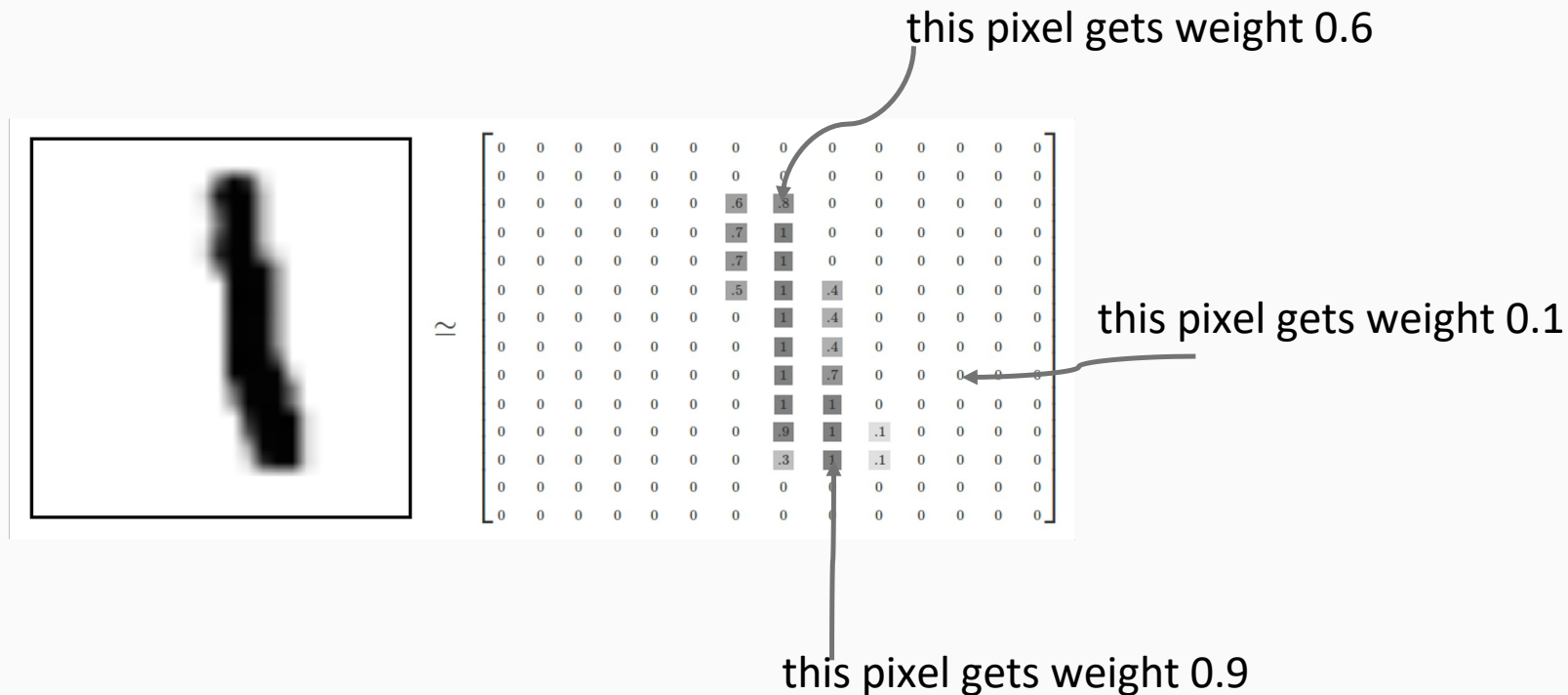
# MNIST dataset



# Encoding Images using MLP



# Is this still a 1? with the shift?

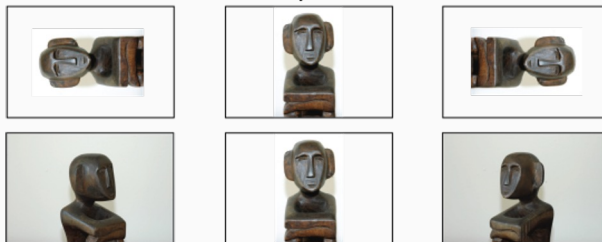


# Translational Invariance

Translation Invariance



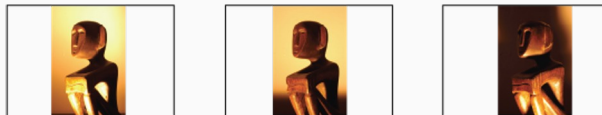
Rotation/Viewpoint Invariance



Size Invariance

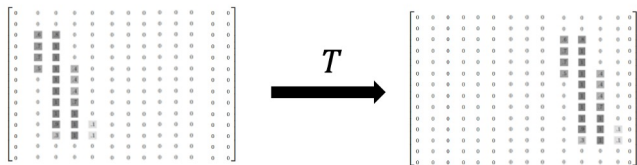


Illumination Invariance



# Translational Invariance

- To make a neural net  $f$  robust in this same way, it should ideally satisfy **translational invariance**:  $f(T(x)) = f(x)$ , where
  - $x$  is the input image
  - $T$  is a translation (i.e. a horizontal and/or vertical shift)

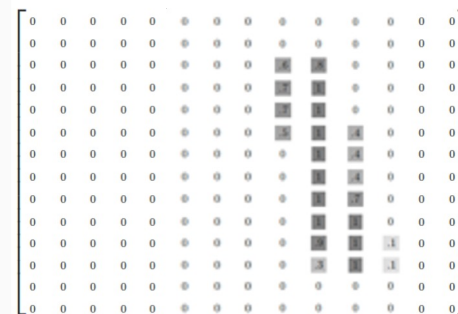
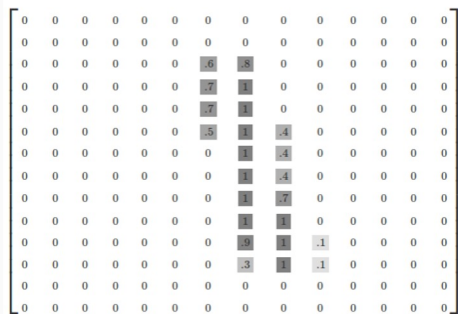


$$f\left(\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}\right) \stackrel{?}{=} f\left(\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}\right)$$

# FCNN are *not* Translationally Invariant

## How to make the network translationally invariant?

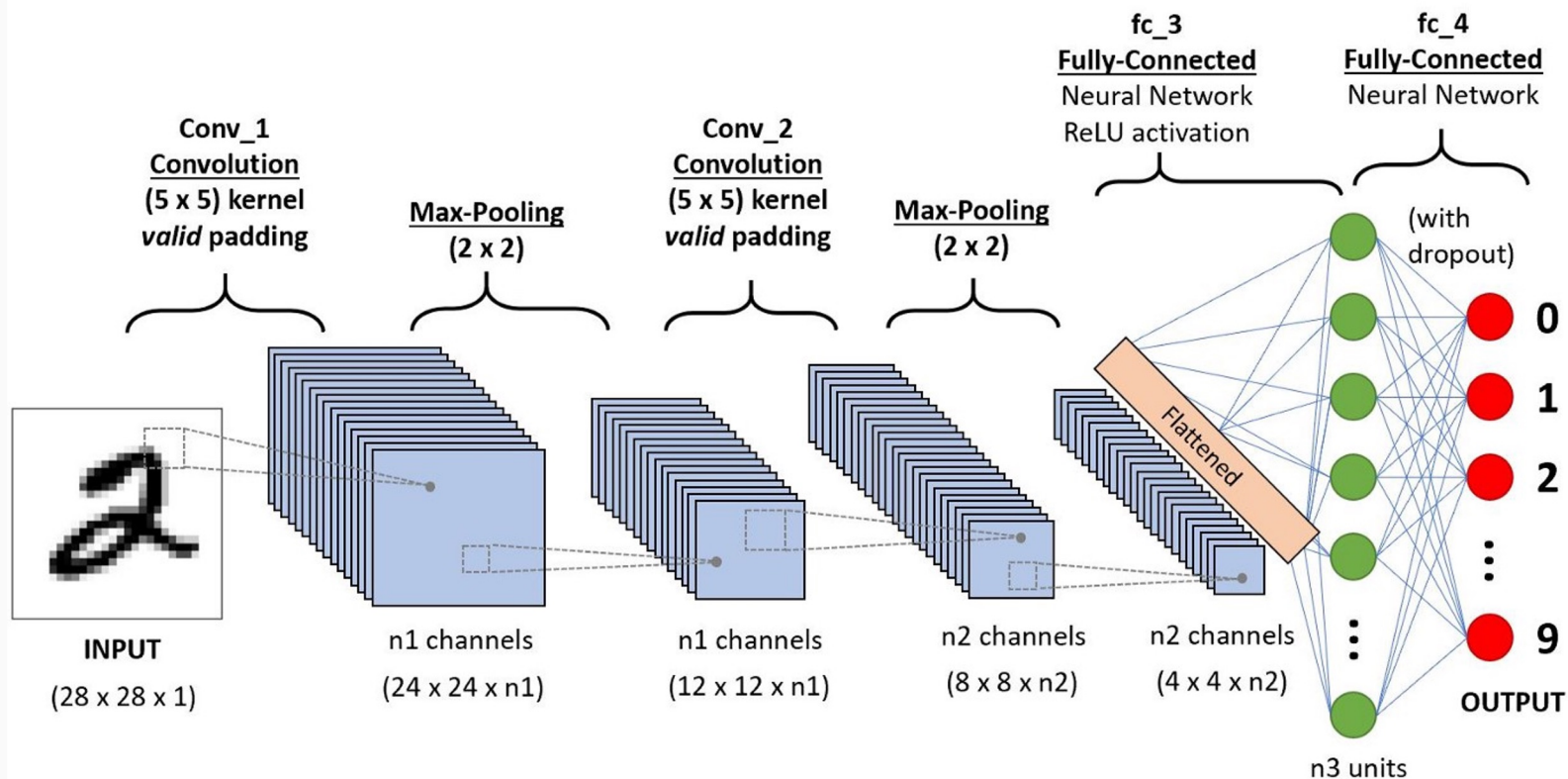
Focus on local differences/patterns



Sum of these three:  $0.6 \cdot 0.8 + 0.1 \cdot 0 + 0.9 \cdot 1 = 1.38$

Sum of these three:  $0.6 \cdot 0 + 0.1 \cdot 0.4 + 0.9 \cdot 0 = 0.4$

# Convolutional Neural Network



# What is a Convolution?

Convolution is an operation that takes two inputs:

(1) An greyscale image (2D – B/W)

(2) A filter (also called a kernel)



|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

2D array of numbers; could be any values

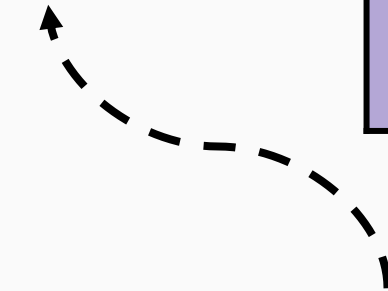
# Convolution Operation

image

|   |   |   |   |
|---|---|---|---|
| 2 | 0 | 1 | 3 |
| 7 | 1 | 1 | 0 |
| 0 | 2 | 5 | 0 |
| 0 | 5 | 1 | 4 |

filter/kernel

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |



(We use this symbol for convolution)  
(The verb form is “convolve”)

# Convolution Operation

|                 |                 |                 |   |   |
|-----------------|-----------------|-----------------|---|---|
| 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 <sub>x1</sub> | 0 | 0 |
| 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 | 0 |
| 0 <sub>x1</sub> | 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 | 1 |
| 0               | 0               | 1               | 1 | 0 |
| 0               | 1               | 1               | 0 | 0 |

Image

|   |  |  |
|---|--|--|
| 4 |  |  |
|   |  |  |
|   |  |  |

Convolved  
Feature

# CNN: Different Kernels

Blur

|       |       |       |
|-------|-------|-------|
| $1/9$ | $1/9$ | $1/9$ |
| $1/9$ | $1/9$ | $1/9$ |
| $1/9$ | $1/9$ | $1/9$ |



Sharpen Kernel

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |

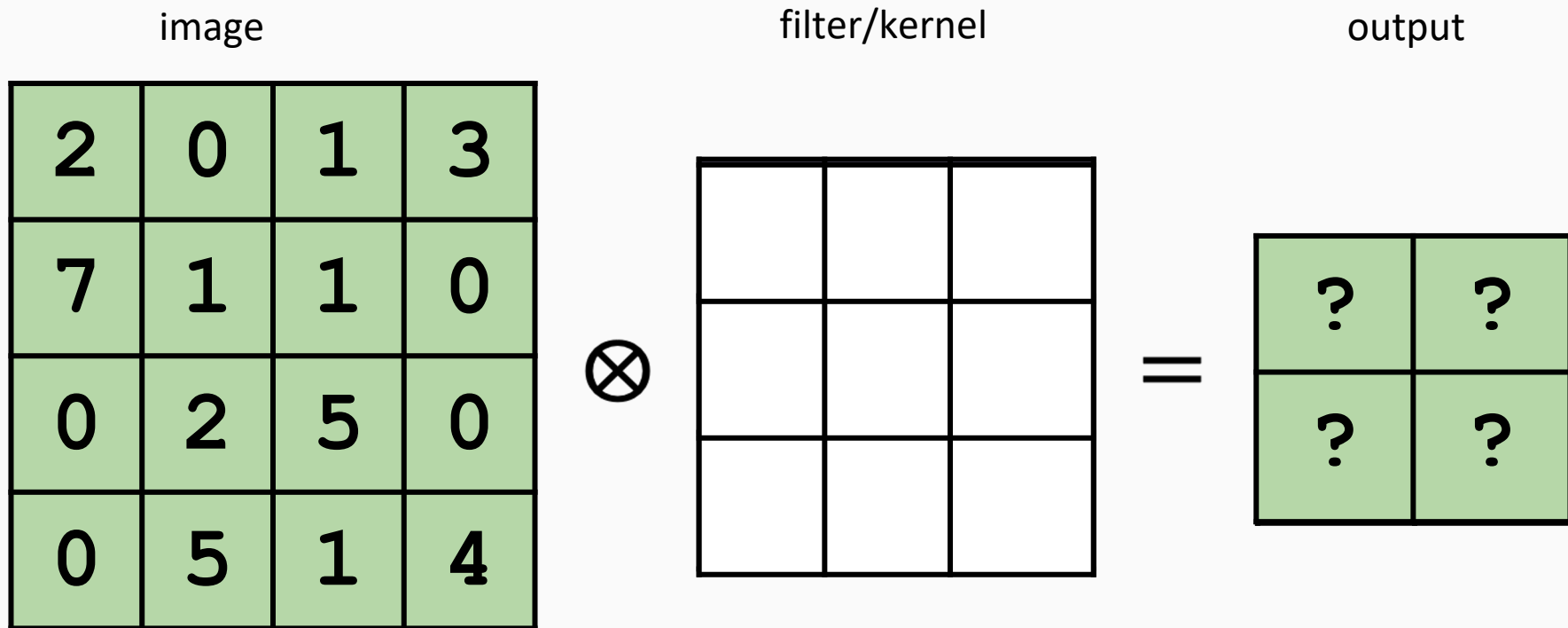


Shift

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 0 |

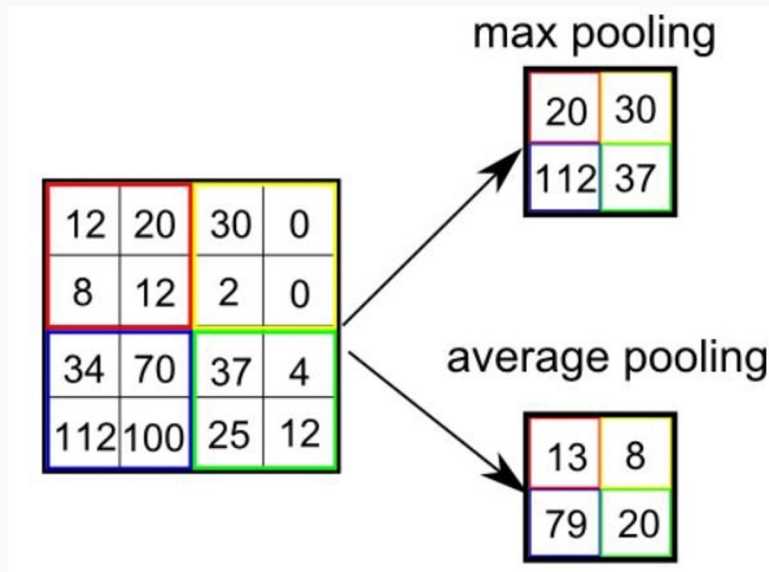


# CNN Kernel is learnable



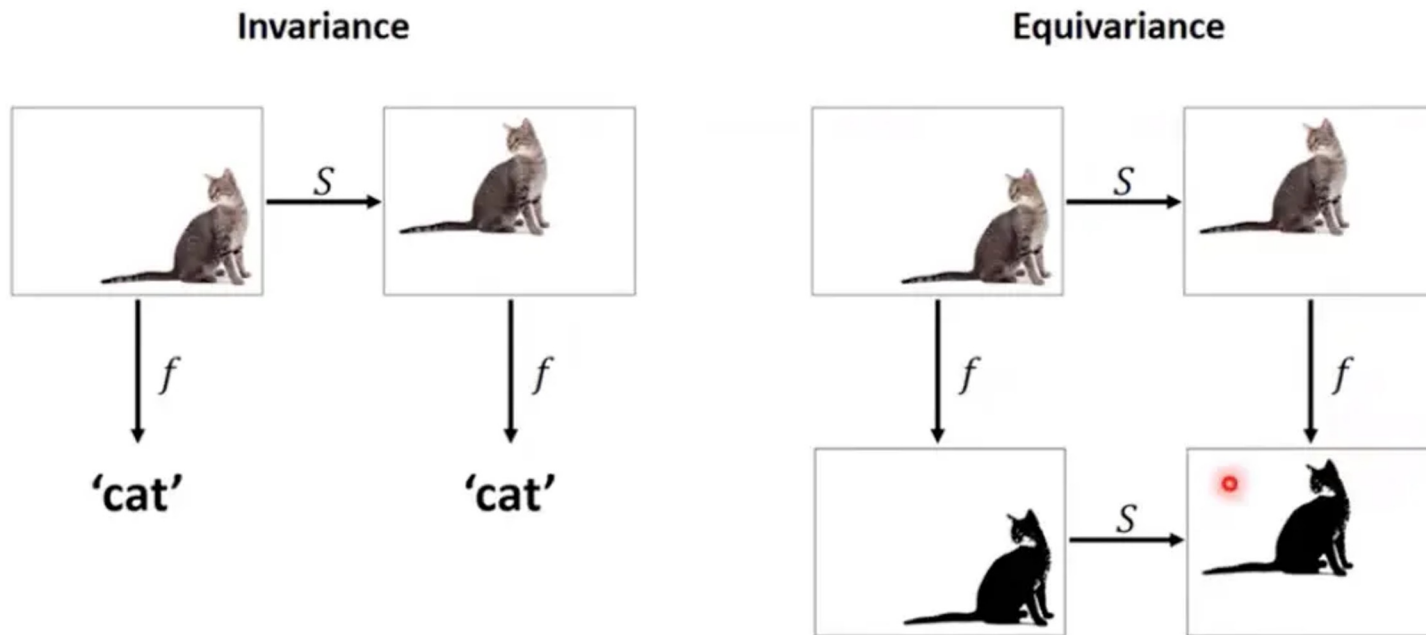
$k_{i,j}$  are learnable parameters

# CNN: Pooling Layer

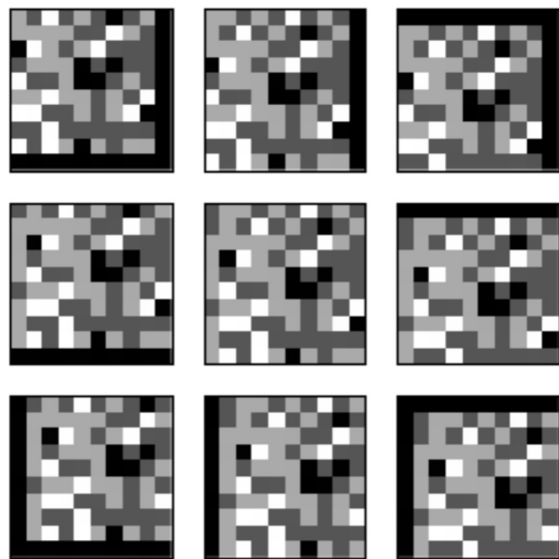


# Pooling and Invariance

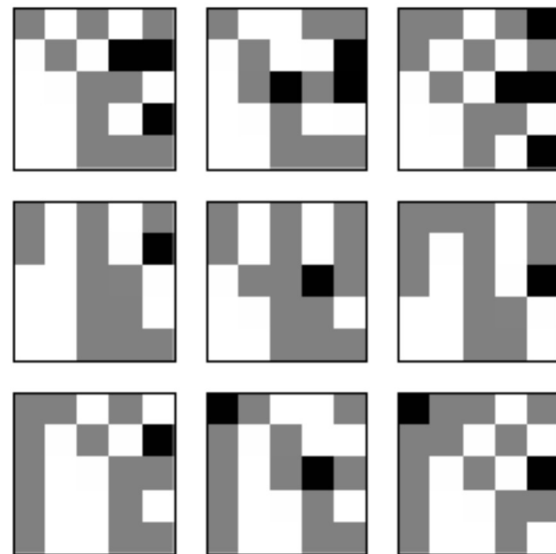
## Invariance vs equivariance



# Are CNN translational invariant?



Max Pool  
→



These are *not*  
all the same!

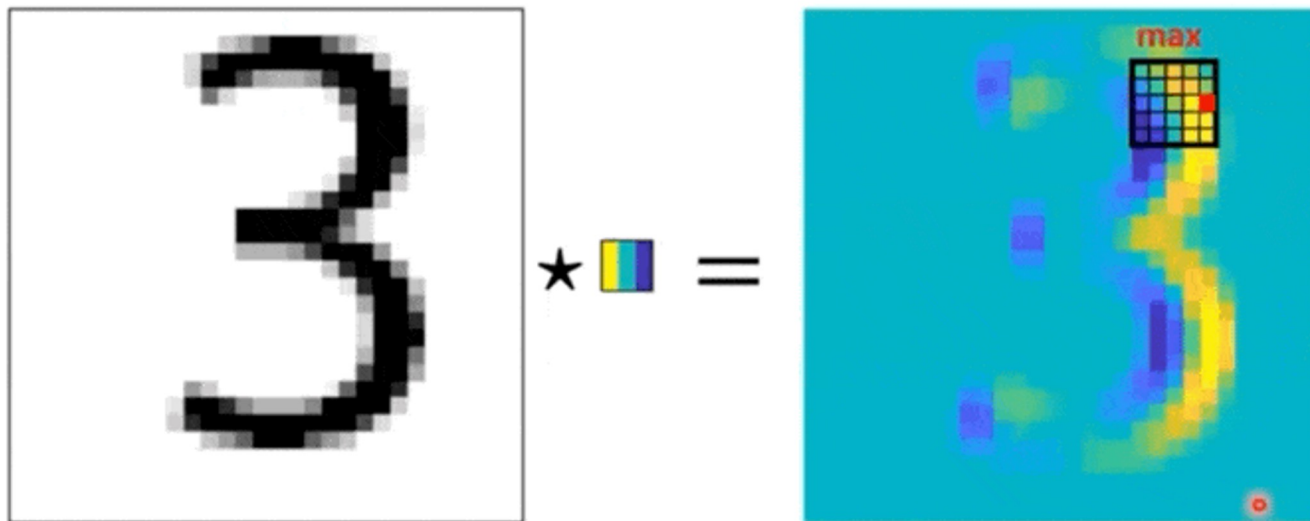
<https://dsp.stackexchange.com/questions/24900/translation-invariance-in-max-pooling-and-cascading-with-convolutional-layer>

Answer: CNNs are “sort of” translation invariant

- Shifting the content of the image around tends not to drastically effect the output classification probabilities...
- It is a local operator. Achieves **partial** invariance to **small** translations.
- ...but they are *not*, strictly speaking, translation invariant

# Shift Invariance in CNN

Approximate invariance in CNNs with pooling



Output of convolutional layer+max pooling (~shift invariant)

# CNN: Kernel & Filter

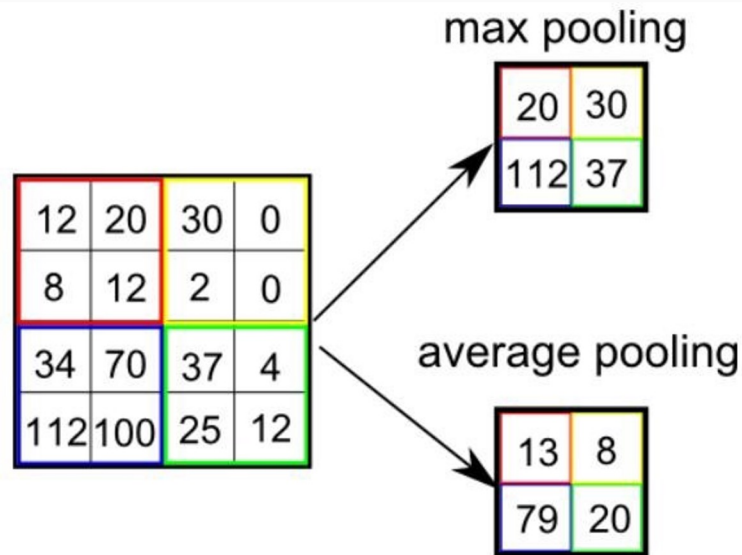
|                 |                 |                 |   |   |
|-----------------|-----------------|-----------------|---|---|
| 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 <sub>x1</sub> | 0 | 0 |
| 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 | 0 |
| 0 <sub>x1</sub> | 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 | 1 |
| 0               | 0               | 1               | 1 | 0 |
| 0               | 1               | 1               | 0 | 0 |

Image

|   |  |  |
|---|--|--|
| 4 |  |  |
|   |  |  |
|   |  |  |

Convolved  
Feature

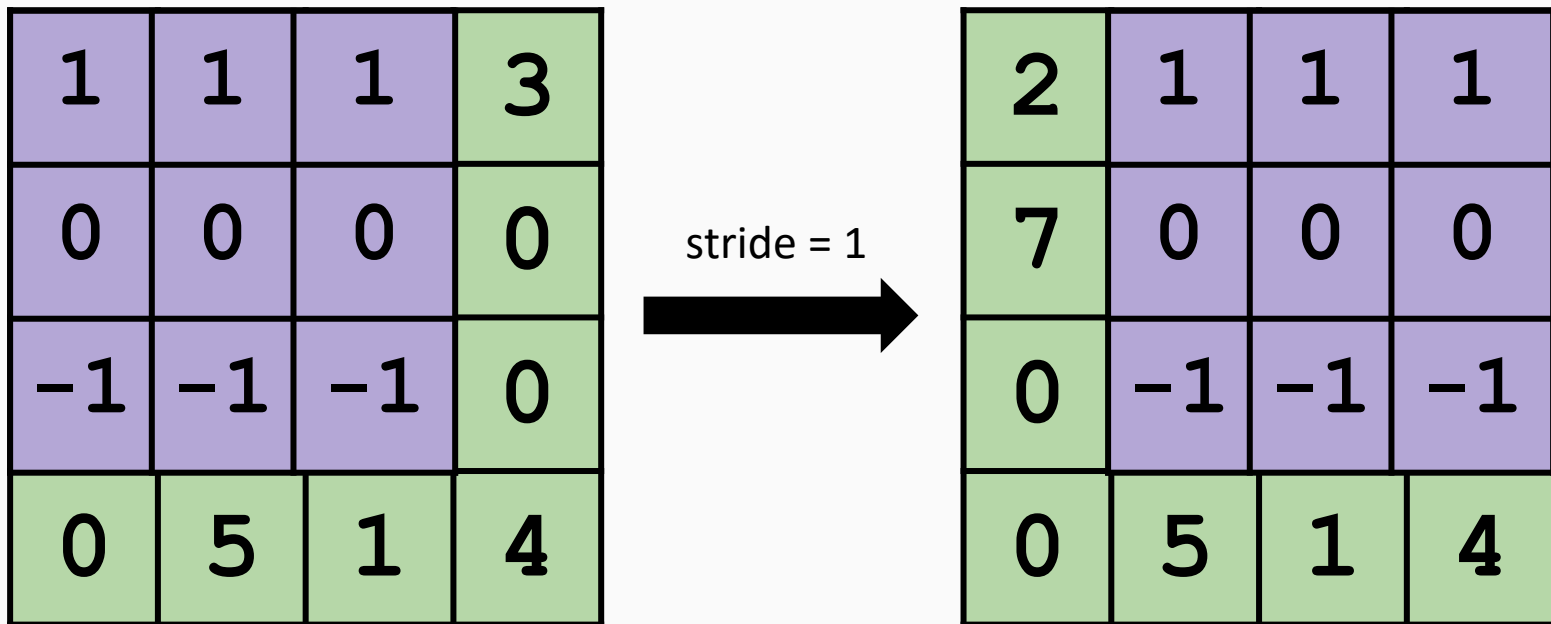
Convolutional Layer



Pooling Layer

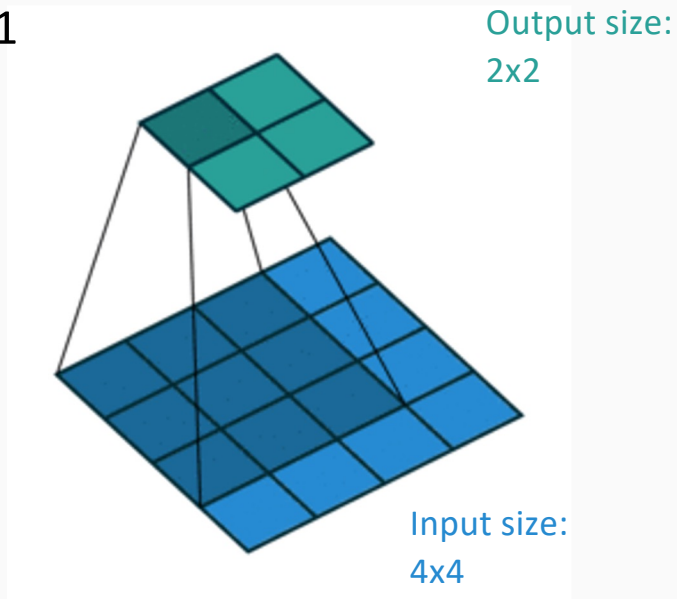
# Stride

- We don't just have to slide the filter by one pixel every time
- The distance we slide a filter by is called ***stride***
  - All the examples we've seen thus far have been stride = 1

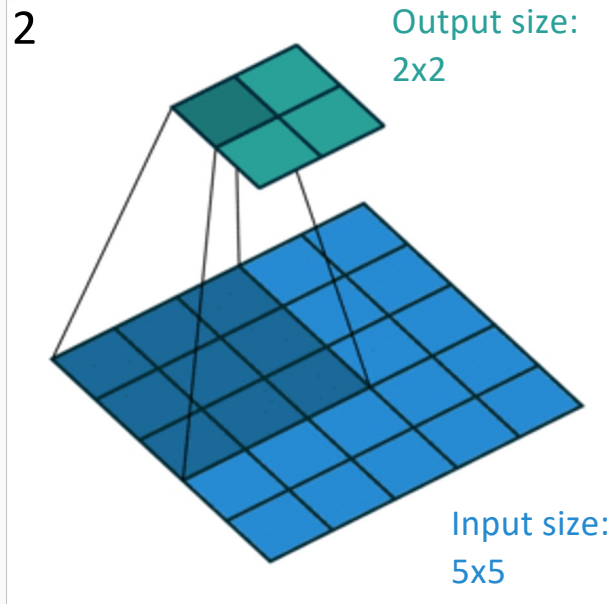


# Stride

Stride: 1



Stride: 2

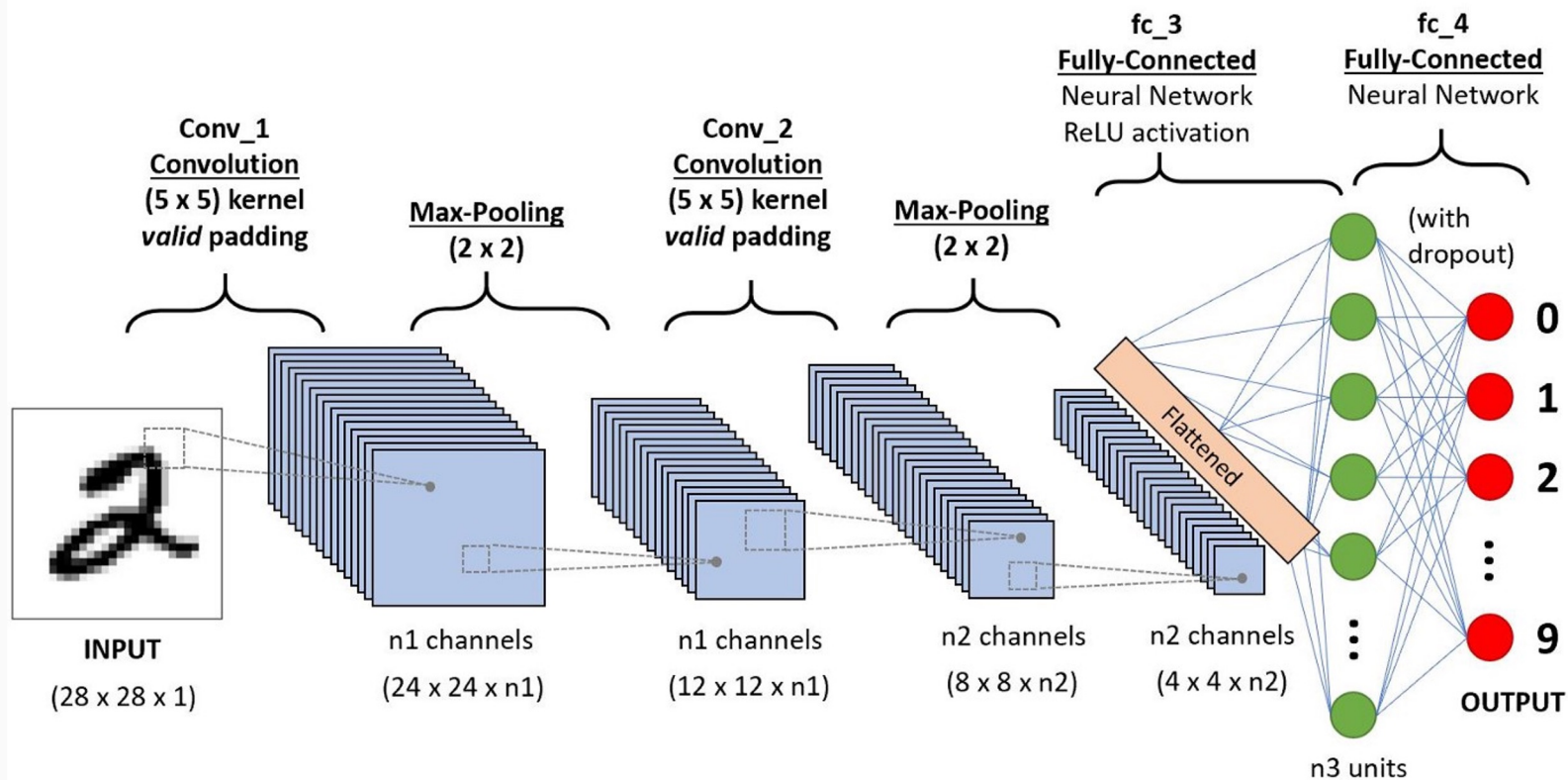


Larger stride turns a bigger input into the same size output

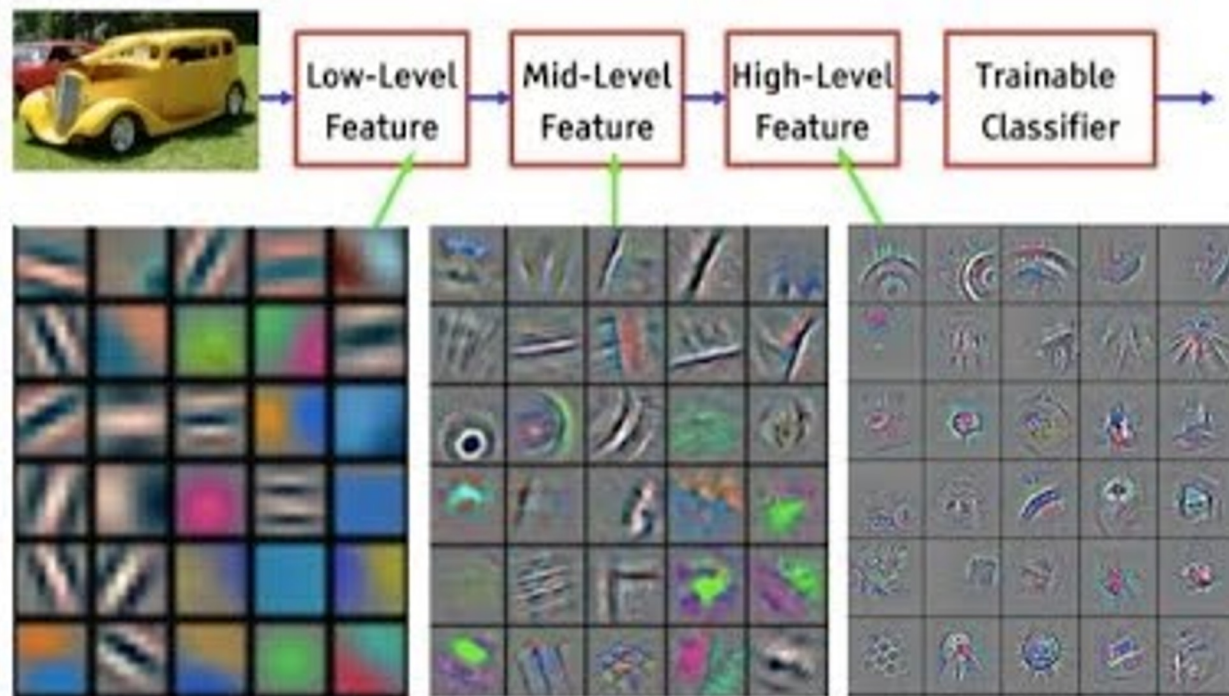
**Corollary:** Larger stride turns the same size input into a *smaller* output

Use this to (controllably) decrease image resolution!

# CNN Architecture

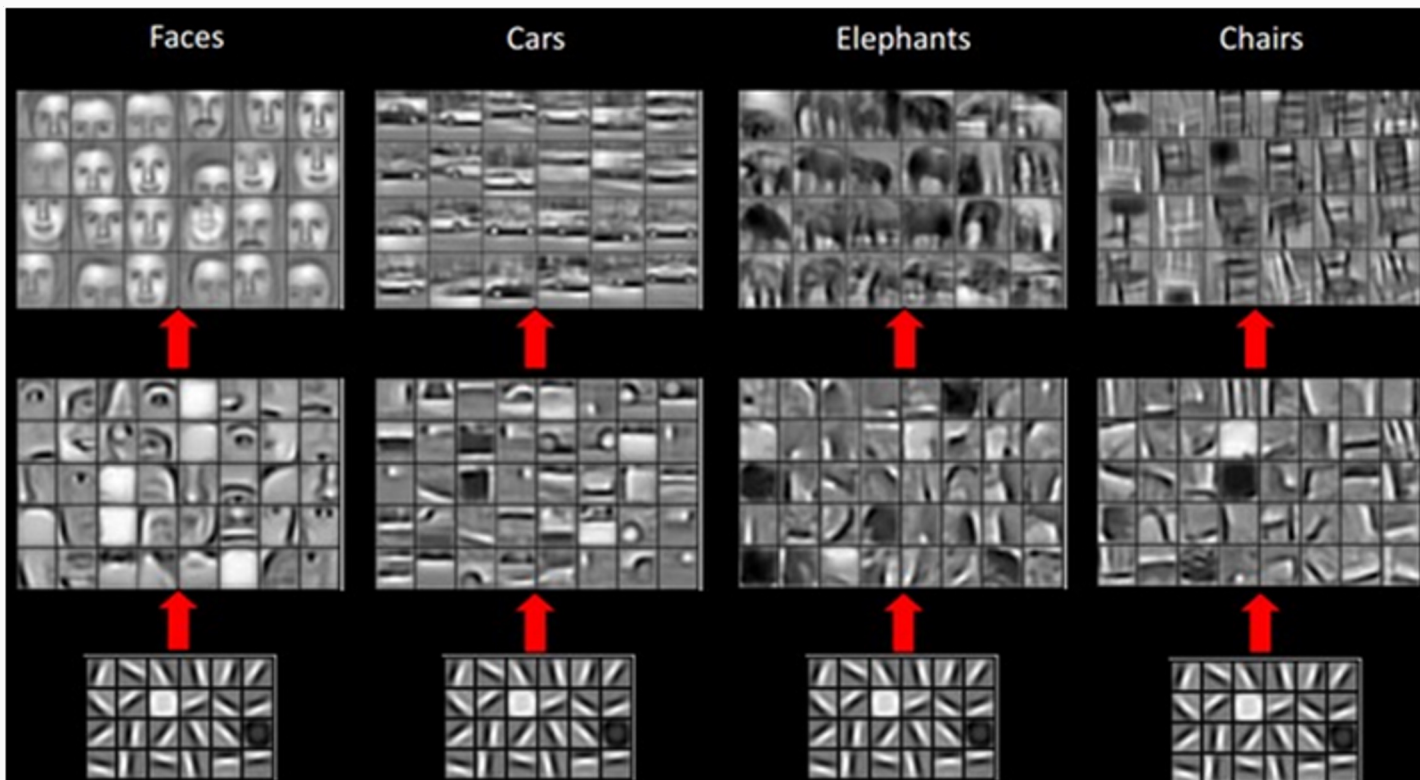


# CNN layers and features



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

# CNN: Learning features



# Padding

## Valid

Filter only slides over  
“Valid” regions of the data

|   |   |   |   |
|---|---|---|---|
| 2 | 0 | 1 | 3 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 2 | 0 |
| 0 | 1 | 1 | 1 |

## Same

Filter slides over the bounds of the data,  
ensuring output size is the “Same” as input  
size (when stride = 1)

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 2 | 0 | 1 | 3 | 0 |
| 0 | 1 | 1 | 2 | 3 | 0 |
| 0 | 4 | 3 | 2 | 1 | 0 |
| 0 | 8 | 3 | 1 | 3 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

# Hands on: Image Classification



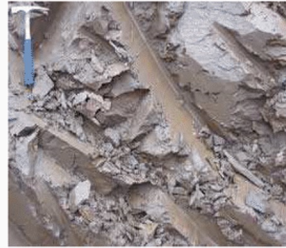
**(a)** Clayey sand



**(b)** Sandy clay



**(c)** Silty sand



**(d)** Clay



**(e)** Humus clay

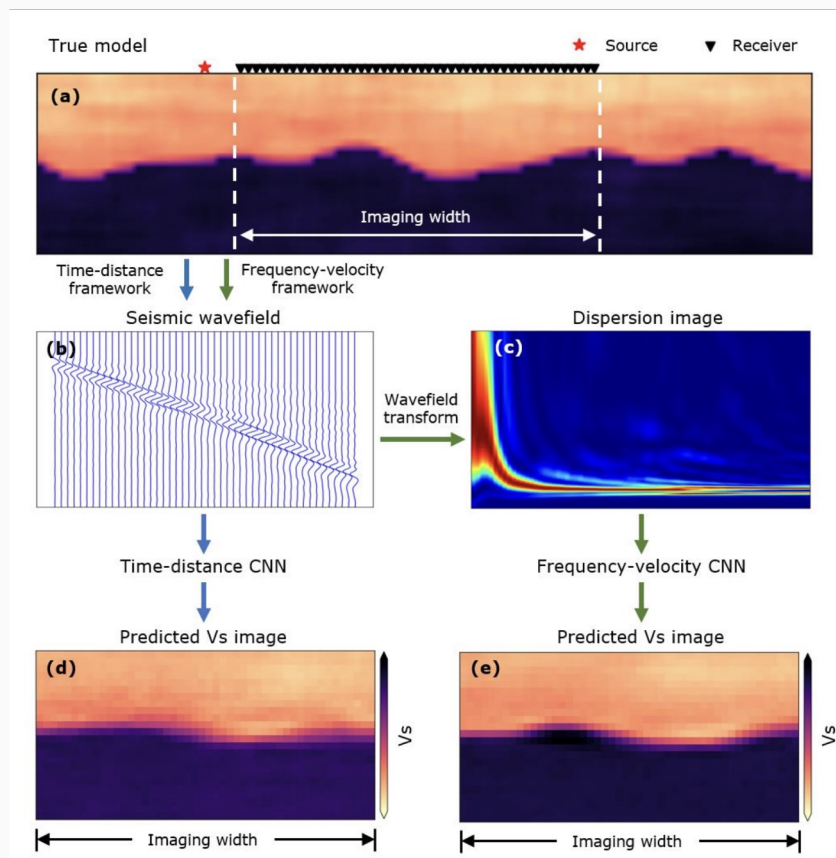


**(f)** Clayey peat



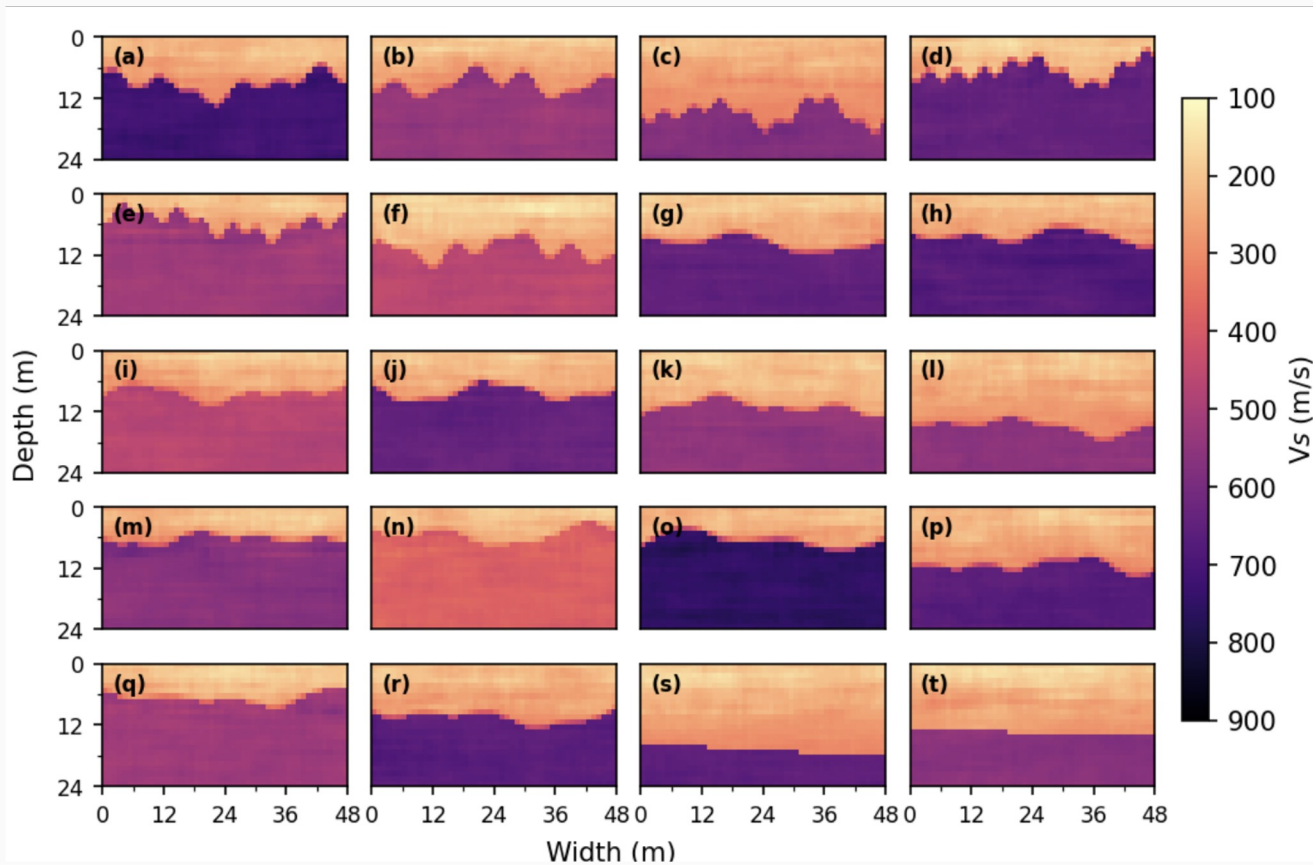
**(g)** Peat

# CNN: Seismic Image to Soil Profile

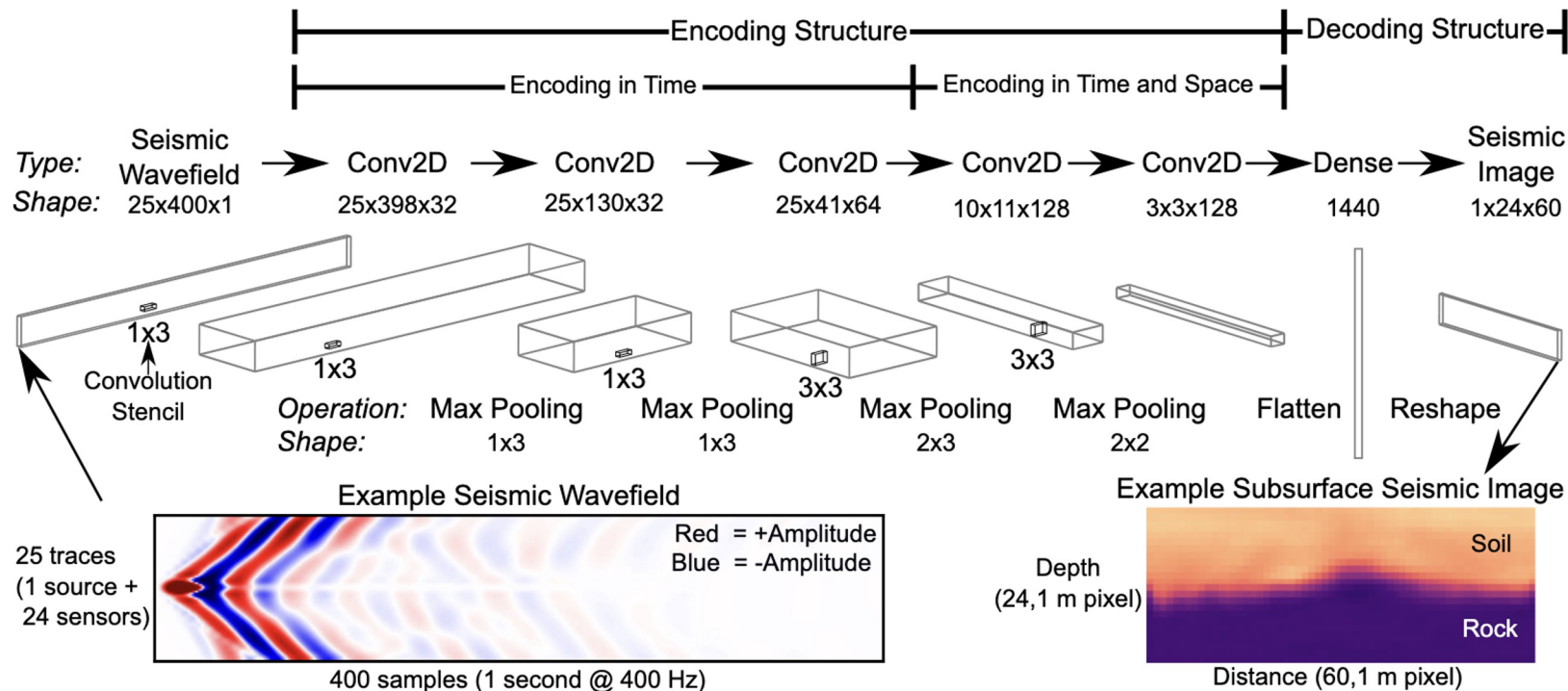


Abbas et al., (2022)

# CNN: FWI



# CNN Architecture



# Building damage classifier - an explanation



Undamaged



Partially-damaged



Completely-destroyed

# Explaining a building image classifier

Sky



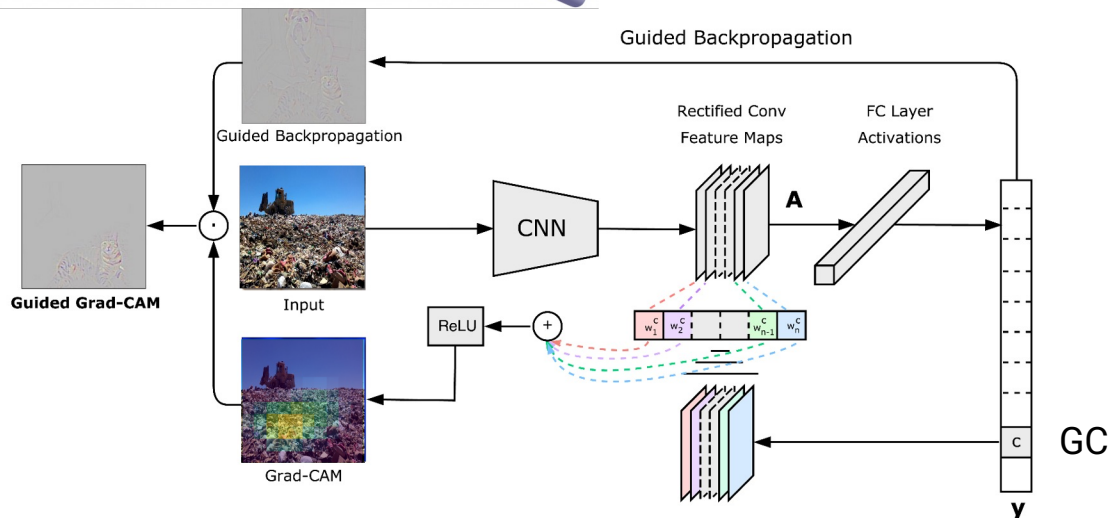
rubble



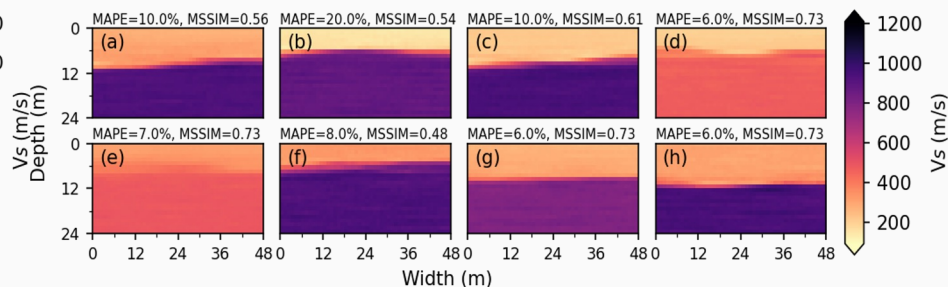
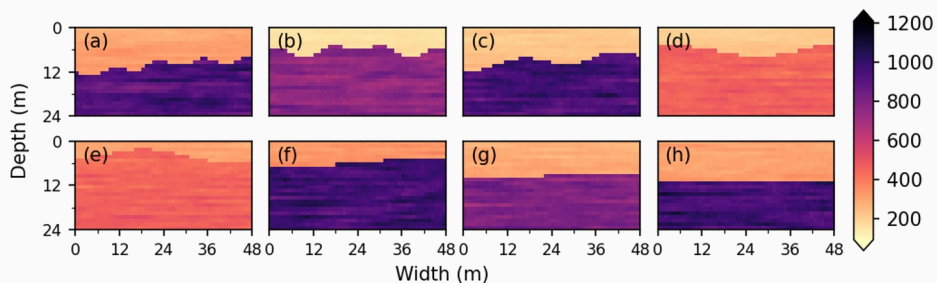
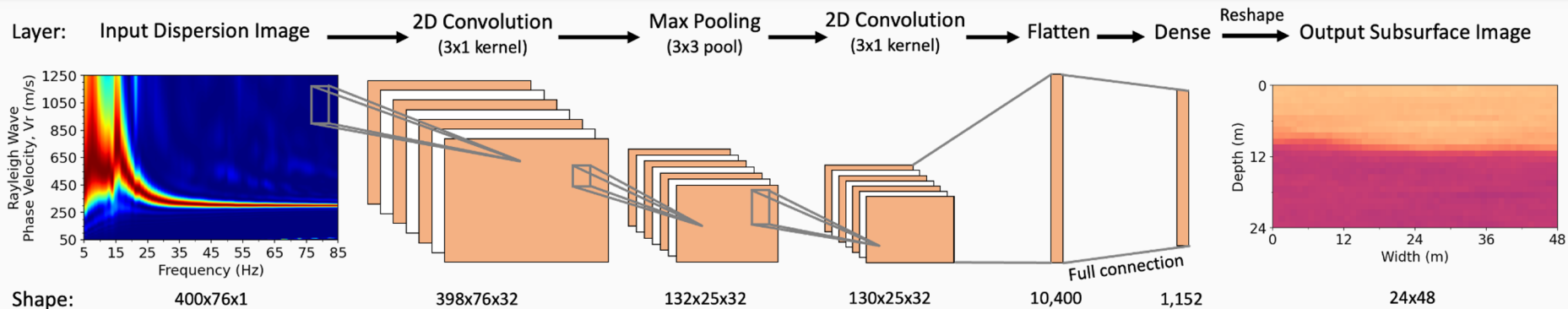
Predicted class: General Collapse

Transfer learning prediction: Ants

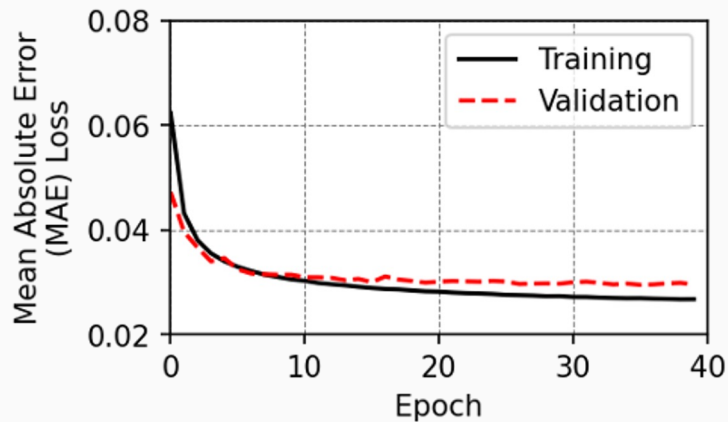
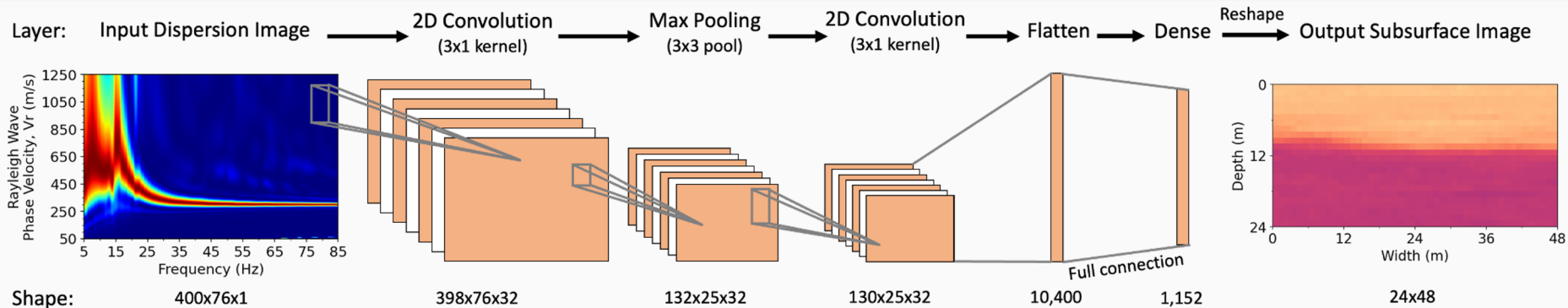
# GradCAM Explanation



# Convolutional Neural Network for inverse problem



# How to design a CNN



| MAPE        | Kernel Size: 3x1 | Kernel Size: 3x3 |
|-------------|------------------|------------------|
| Deep (5)    | 11.3 %           | 8.5 %            |
| Shallow (3) | 12.1 %           | 11.2 %           |

# Designing a neural network with explainability

